FPGA Placement with Simulated Annealing: Approach Report

1. Overview

This project implements a simulated annealing (SA) based placer for FPGA logic blocks and IOs. The placer aims to minimize total wirelength and congestion, efficiently handling large netlists and grid sizes by leveraging incremental update techniques and careful cost function design.

2. Simulated Annealing Cost Function Design and Parameters

The cost function used in the SA process is a weighted combination of total wirelength and congestion cost:

$$Cost = \alpha \cdot \frac{\Delta \text{HPWL}}{\text{HPWL}_{\text{prev}}} + \beta \cdot \frac{\Delta \text{CC}}{\text{CC}_{\text{prev}}}$$

Where: - HPWL is the sum of half-perimeter wirelengths of all nets. - CC is a congestion metric based on the linear and quadratic sums of the cell coverage map U. - α and β are scaling factors.

Parameter values used: - Initial temperature $T_0 = 800,000$ - $\alpha = 100 \times T_0$ - $\beta = 100 \times T_0$ - Cooling interval: every 1000 iterations - Cooling rates: 0.92 (acceptance rate > 0.95), 0.99 (> 0.8), 0.995 (> 0.5), 0.999 (otherwise) - Termination: after 235 seconds

The cost function is normalized to allow fair comparison between moves and to balance the impact of wirelength and congestion.

3. Temperature Tuning Logic

The temperature schedule is adaptive and dynamically controlled based on the acceptance rate of moves:

Initialization: - The initial temperature T_0 is set to a high value (800,000) to allow broad exploration.

Cooling Schedule: - Every 1000 iterations, the acceptance rate (ratio of accepted moves to attempted moves) is measured. - The cooling rate is chosen according to the following logic:

```
if (acceptRate > 0.95)
    coolingRate = 0.92;
else if (acceptRate > 0.8)
    coolingRate = 0.99;
else if (acceptRate > 0.5)
```

```
    coolingRate = 0.995;
else
    coolingRate = 0.999;
T = T * coolingRate;
```

- This means that if the system is accepting many moves (high accept rate), the temperature drops quickly to encourage convergence. If the system is stuck (low accept rate), the temperature cools more slowly to allow more exploration.
- The process terminates after a fixed time limit (235 seconds).

4. Fast Incremental Updates for Nets, U, and Congestion Costs

To achieve high performance, the code uses several incremental update techniques:

4.1. Net Bounding Box Updates

- Each net maintains its bounding box (x_{min}, x_{max}, y_{min}, y_{max}).
- When a block is moved or swapped, only the nets connected to the affected blocks are considered.
- For each affected net, a fast anchor check is performed:
  - If the moved block was on the bounding box edge, a scan is performed to find the new min/max among remaining blocks and IOs.
  - Otherwise, the bounding box is updated in $O(1)$ time.
- All changes are recorded in a `NetDiff` structure, allowing efficient rollback if a move is rejected.

Pseudocode:

```
for each affected net:
        save old bounding box (x_min, x_max, y_min, y_max)
        update bounding box with new block positions:
                if moved block extends box:
                        x_min = min(x_min, new_x)
                        x_max = max(x_max, new_x + 1)
                        ... (same for y)
                if moved block was on box edge:
                        scan all connected blocks and IOs to find new min/max
        record new bounding box in NetDiff
```

4.2. U Grid and Congestion Cost Updates

- The cell coverage map U is updated incrementally:
  - For each affected net, the old bounding box region is decremented and the new region is incremented.
- Congestion costs (CC_{linear} and CC_{square}) are updated in place as U changes, avoiding full recomputation.

- Rollback is supported by applying the inverse of the recorded diffs.

Pseudocode:

```
for each NetDiff in net_diffs:
        // Remove old coverage
        for x in ceil(old_x_min) to floor(old_x_max)-1:
                for y in ceil(old_y_min) to floor(old_y_max)-1:
                        CC_linear -= U[x][y]
                        CC_square -= U[x][y] * U[x][y]
                        U[x][y] -= 1
        // Add new coverage
        for x in ceil(new_x_min) to floor(new_x_max)-1:
                for y in ceil(new_y_min) to floor(new_y_max)-1:
                        U[x][y] += 1
                        CC_linear += U[x][y]
                        CC_square += U[x][y] * U[x][y]
```

4.3. Efficient Data Structures

- Static buffers (`static vector<int> affected_nets` and `static vector<NetDiff> net_diffs`, see `SA_update.cpp`) are used for affected nets and net diffs to avoid repeated allocations.
- All updates and rollbacks are performed in-place, minimizing overhead and maximizing cache efficiency.

5. Review

Having prior experience on SA placers, I know that the cost function design and incremental updates are critical for performance. This implementation focuses on these aspects to ensure efficient perturbation evaluation.

However, I have been busy in recent weeks, and incremental updates are inevitably more complex to implement correctly and it took me quite a lot of effort. As a result, when I started working on cal_net_update and update_congestion_cost, I was almost out of time,forcing me to finish them with the help of Gemini. While it works just fine, I noticed that it tend to overestimate the cost difference. This may lead to suboptimal moves being rejected more often than they should, potentially impacting the overall placement quality. Unfortunately, I did not have enough time to check and refine this part before the deadline.

References

1. "A Congestion Driven Placement Algorithm for FPGA Synthesis", In Proceedings of the International Conference on Field Programmable Logic and Applications (FPL 2006)

2. Vaughn Betz and Jonathan Rose. 1997. VPR: A new packing, placement and routing tool for FPGA research. In Proceedings of the 7th International Workshop on Field-Programmable Logic and Applications (FPL '97). Springer-Verlag, Berlin, Heidelberg, 213–222.